

Your 1-Month E-Commerce Roadmap

1–2 hrs/day · ~45 hrs total · Full understanding, zero shortcuts · Deployed by Day 30

The honest truth: The full project = ~400 hrs. At 1–2 hrs/day you have ~45 hrs. So Month 1 builds a real, deployed, impressive MVP — product browsing, auth, cart, orders. Stripe, WebSockets, Docker, Kubernetes, CI/CD go in Months 2 & 3. You will understand every single line we write.

30	~45	4	1
Days	Total hrs	Phases	Live app

Week 1 — Foundations Days 1–7

React · TypeScript · Next.js · Git

Day 1	How the web actually works	HTTP requests, client vs server, what a browser does, why React exists. Zero code — pure mental model. This day saves you 10 confused days later.
Day 2	React fundamentals	What a component is, JSX syntax, props, returning HTML from JS. Build your first 3 components: Button, Badge, ProductCard (static).
Day 3	TypeScript — why types matter	Primitive types, interfaces, why TypeScript catches bugs before you run the code. Convert components to TypeScript. Define the Product type.
Day 4	Next.js setup — the full project	Why Next.js over plain React (SSR, routing, performance). Create the project, understand folder structure, file-based routing. Pages: home, /products.
Day 5	Tailwind CSS — styling that scales	Utility-first CSS, responsive classes (sm: md: lg:). Build Header, Footer, Navbar. Make the product grid responsive. No more fighting CSS files.
Day 6	Git & GitHub — how pros work	git init, commit, push, branches, pull requests. Set up your GitHub repo. This becomes your portfolio link. Write a proper README.
Day 7	Review + build day	Wire everything together. Static home page with hero, product grid (hardcoded data), working navigation. You can see the site in your browser.

✓ A running Next.js + TypeScript project · ✓ Home page + Products page with static data · ✓ Full understanding of React components & Tailwind · ✓ GitHub repo set up and pushed

Week 2 — Frontend core Days 8–14

Hooks · Redux · Forms · Dynamic routes

Day 8	React hooks deep dive	useState (local state), useEffect (side effects & lifecycle), the mental model of re-rendering. Build a counter, then a live search filter.
Day 9	Redux Toolkit — global state	Why global state exists (prop drilling problem). Actions, reducers, slices. Set up store, cartSlice, authSlice. useSelector & useDispatch.
Day 10	Shopping cart — full feature	Add to cart, remove, update quantity, calculate totals. CartPage component. Persist to localStorage with a custom hook. Your first real feature.

Day 11	Dynamic routing — product detail page	Next.js [id] routes, useRouter hook, getStaticPaths vs getServerSideProps. Build the full product detail page: images, description, add-to-cart.
Day 12	Forms — login & register	Controlled inputs, Formik for form state, Yup for validation rules. Build login form, register form, password reset. Client-side validation.
Day 13	API layer — axios & services	How the frontend talks to a backend. Axios instance, base URL, interceptors. Build productService, authService. useFetch custom hook.
Day 14	Review + polish day	Loading states, error states (ErrorBoundary), empty states. Product filtering by category. Search with debounce. Make every page feel finished.
✓ Fully interactive product browsing + detail pages · ✓ Working shopping cart with localStorage persistence · ✓ Login & register forms with validation · ✓ Clean frontend architecture you can explain in an interview		

Week 3 — Backend core Days 15–21		Express · PostgreSQL · Prisma · REST API
Day 15	Node.js & Express — your first server	What a server actually does. HTTP methods (GET POST PUT DELETE). Express routing, request & response objects. Build: GET /api/products returns JSON.
Day 16	Architecture — controllers, routes, services	Why we separate concerns (routes → controllers → services → repository). TypeScript in Express. Middleware concept. Build the pattern for all future endpoints.
Day 17	PostgreSQL & Prisma — real data	Relational databases, tables, foreign keys, why SQL exists. Prisma schema, migrations, Prisma Studio. Set up User + Product + Category models.
Day 18	Product & Category CRUD	Full CRUD endpoints: list products, get one, create, update, delete. Pagination query params. Filter by category. Seed the DB. Test in Postman.
Day 19	Middleware — auth, validation, errors	JWT authentication middleware, express-validator for input validation, global error handler with AppError class. Helmet + CORS + rate limiting.
Day 20	Cart & Order endpoints	Cart schema (Cart → CartItems → Products), cart CRUD. Order schema with status enum. Create order from cart, order history. Full Prisma relations.
Day 21	Review + Postman testing day	Test every single endpoint in Postman. Write a Postman collection (this IS documentation). Fix bugs. Add proper status codes and error messages.
✓ A real Express + TypeScript backend server · ✓ PostgreSQL database with full schema · ✓ All major API endpoints (products, cart, orders, users) · ✓ Auth middleware, validation, error handling — production patterns		

Week 4 — Integration & launch Days 22–30		Auth · Full-stack connection · Deploy
Day 22	Auth system — register & login	bcrypt password hashing (why you NEVER store plain passwords), JWT token generation, register & login endpoints, refresh token strategy.
Day 23	Frontend auth — the full flow	Connect login/register forms to backend. Store JWT in httpOnly cookie. Auth context + useAuth hook. ProtectedRoute HOC. Redirect unauthenticated users.

Day 24	Connect all frontend to backend	Replace all mock data with real API calls. Environment variables (NEXT_PUBLIC_API_URL). Axios interceptors for auth headers. Handle loading & error states.
Day 25	Cart sync to database	When user logs in, sync localStorage cart to DB cart. Cart persists across devices. Merge carts if user adds items before login. Full cart feature — done.
Day 26	Checkout flow + order history	Shipping address form → order review → place order (mock payment for now). Order created in DB. User dashboard showing order history with status.
Day 27	Deployment day	Frontend → Vercel (free, auto-deploys on push). Backend + DB → Railway (free tier, PostgreSQL included). Set env variables. Your app is live on the internet.
Day 28	Documentation & Swagger	Swagger/OpenAPI setup for your backend. Write the README properly (screenshots, setup instructions, tech stack, architecture). What interviewers read first.
Day 29	Polish & bug fixes	Go through every page as a real user. Fix broken flows. Add toast notifications. Improve loading states. Fix mobile responsiveness. Make it feel finished.
Day 30	Final walkthrough + prep	Can you explain every file? Write down 10 things you learned. Prepare to talk about your architecture decisions. You are ready for technical interviews.

✓ Product catalog — browse, filter, search, detail pages · ✓ User auth — register, login, JWT, protected routes · ✓ Shopping cart — full CRUD, synced to database · ✓ Checkout + order history — full flow, orders in DB · ✓ Live URL on Vercel + Railway for your portfolio · ✓ Swagger API docs + proper README

After Month 1 — what comes next

Month 2 • Stripe payment integration • Product search (Elasticsearch) • Product reviews & ratings • Admin dashboard • Redis caching • Email notifications	Month 3 • Docker + docker-compose • CI/CD with GitHub Actions • WebSockets (real-time updates) • Unit + integration tests • Kubernetes basics • Performance optimization
--	---